

INSTITUT SUPERIEUR D'INFORMATIQUE
ET DES TECHNIQUES DE COMMUNICATION – SOUSSE
لمعهد العالي للإعلامية وتقنيات الاتصال بسوسة



Rapport de stage de fin d'études

Présenté en vue de l'obtention du diplôme de Licence en Science de
l'Informatique (Computer Science)

Spécialité : Informatique et Multimédia

Conception et Développement d'une application web\mobile Semi réseau professionnelle innovante

Réalisé par :
Cyrine Hanzouti
Tarek Dhokkar

Encadrant académique : : Mr Abdelweheb Gueddass

Encadrant professionnel : Mr Mohamed Ben Ameur

Société d'accueil



Année universitaire : 2024/2025

Institut Supérieur d'Informatique et des Techniques de Communication Sousse **ISITCOM**
Tél/Fax : +216 73 37 15 71 / +216 73 36 44 11

Dédicaces

Je dédie ce travail à toutes les personnes qui occupent une place précieuse dans ma vie, et sans qui cette étape n'aurait pas été possible.

À ma mère, mounira ayed

À ma mère, mon refuge et ma force. Ce travail est le fruit de tes sacrifices silencieux et de ta patience infinie, piliers inébranlables de mon parcours. Si chaque réussite porte l'éclat de ta lumière, ce projet te revient tout particulièrement. Plus qu'une mère, tu es mon inspiration et mon ange gardien. Reçois ce témoignage de ma profonde gratitude et de mon amour éternel

À mon père, ibrahim selmi

À mon père, dont la sagesse et la confiance ont été mes guides les plus sûrs. Ton soutien indéfectible et ta présence discrète ont constitué le roc sur lequel j'ai bâti ce projet. En m'enseignant la valeur du travail, l'honneur et le courage, tu m'as donné les clés de ma propre réussite. Ce travail est le reflet de ton héritage et de ton amour.

À ma soeur ,farah

Ta douceur, ta réussite et ton amour discret m'ont porté plus que tu ne peux l'imaginer. Merci d'avoir été mon repère et mon inspiration silencieuse.

À mon frère,ahmed

À mon frère et ami de toujours. Ton énergie et ta joie de vivre sont de précieux cadeaux. J'ai une confiance absolue en ta réussite future, et particulièrement pour ce baccalauréat que tu t'apprêtes à conquérir. Je suis de tout cœur avec toi.

À ma famille

À ma famille, mon premier ancrage et ma plus grande fierté. Merci d'avoir été mon sanctuaire tout au long de ce parcours. Votre amour inconditionnel et votre solidarité exemplaire ont été le souffle nécessaire pour mener ce projet à son terme. Ce travail est le vôtre, car il est le fruit de tout ce que vous m'avez transmis.

À mes amis

À mes précieux amis, piliers indispensables de cette aventure. Votre écoute et votre générosité ont transformé les moments de doute en force. Merci d'avoir été mon ancrage et de m'avoir soutenu avec autant de fidélité. Ce travail vous doit aussi beaucoup.

À mes collègues

À cette équipe exceptionnelle qui a été à mes côtés à chaque instant. Merci pour votre soutien sans faille et cette énergie collective qui a rendu ce projet non seulement possible, mais passionnant. Votre présence a été le moteur de ma motivation, rendant chaque étape plus fluide et enrichissante.

À vous tous, merci du fond du cœur. Ce projet porte en lui l'empreinte de votre affection et la force de vos encouragements. Il est autant le fruit de mon labeur que celui de votre amour et de votre soutien constant

Remerciements

Le plus dur n'est pas de rédiger le rapport, mais de remercier toutes les personnes qui nous ont soutenus dans la réalisation de ce projet.

Je tiens à exprimer mes plus sincères remerciements à **Mme Hinda Boudogga**, mon encadrante académique, pour son accompagnement rigoureux et ses conseils éclairés tout au long de ce stage. Sa rigueur scientifique et sa disponibilité constante ont été déterminantes dans la réussite de ce projet. Grâce à son expertise et à ses retours constructifs, j'ai pu approfondir mes connaissances et consolider mon approche méthodologique .

Nos remerciements s'adressent également à **M. Tarek Dhokkar**, notre encadrant professionnel, pour son accompagnement et ses conseils précieux. Nous le remercions pour son implication et pour la confiance qu'il nous a témoignée tout au long de ce parcours .

Nous remercions **M. Hichem Chaaben**, Directeur de l'Institut, pour les excellentes conditions d'apprentissage mises à notre disposition. Notre reconnaissance va aussi à **M. Slim Abdelbari**, Chef de Département, pour son accompagnement et son suivi tout au long de notre parcours.

Nos remerciements vont également à **M. Zied Jday**, Directeur de l'entreprise, pour nous avoir accueillis au sein de sa structure. Nous le remercions pour l'environnement professionnel stimulant et favorable à l'apprentissage qu'il a su instaurer, nous permettant ainsi d'évoluer dans les meilleures conditions.

Nous tenons à exprimer notre profonde reconnaissance à toute l'équipe pédagogique de l'**ISITCOM**. Nous remercions tout particulièrement nos professeurs pour leur accompagnement constant tout au long de notre parcours, ainsi que pour les précieux conseils qu'ils nous ont prodigués avec dévouement.

Aux membres du jury : nous vous exprimons notre vive gratitude pour l'honneur que vous nous faites en acceptant de juger ce travail. Veuillez recevoir, à travers ce mémoire, l'expression de notre haute considération et de nos remerciements les plus sincères.

Nous remercions enfin toutes les personnes qui nous ont aidés ou soutenus, de quelque manière que ce soit, dans l'élaboration de ce projet.

Table des matières

Introduction Générale	1
1 Présentation générale du projet	2
1.1 Introduction	2
1.2 Organisme d'accueil	2
1.2.1 Présentation de société	2
1.2.2 Les Services	3
1.3 Etude préalable	3
1.3.1 Contexte de projet	3
1.3.2 Problématique	3
1.3.3 Étude de l'existant	4
1.3.4 Etude comparative	7
1.3.5 Solution proposée	8
1.4 Méthodologie de travail	8
1.4.1 Framework Scrum	9
1.4.2 Processus Scrum	9
1.4.3 Les avantages de Scrum	10
1.4.4 Les acteurs de scrum	11
1.4.5 Les artefacts de scrum	11
1.5 Planification itérative du projet	12
1.5.1 Langage de modélisation UML	12
1.6 Conclusion	12
2 Architecture, outils et environnement de développement	14
2.1 Introduction	14
2.2 Capture des besoins	14
2.2.1 Identification des acteurs	14
2.2.2 Besoins fonctionnels	15
2.2.3 Besoins non fonctionnels	16
2.3 Product Backlog	16

2.4	Les Diagrammes	21
2.4.1	Diagramme de cas d'utilisation global	21
2.4.2	Diagramme de classes global	24
2.5	Architecture de l'application	26
2.6	Outils et environnement de développement	29
2.6.1	Environnement matériel	29
2.6.2	Environnement technique	29
2.7	Conclusion	30
Conclusion Générale		31
2.8	Synthèse du Projet	31
2.9	Récapitulatif des Réalisations	31
2.10	Architecture Technique	32
2.11	Technologies et Innovations	33
2.12	Défis Relevés	33
2.13	Perspectives d'Évolution	33
2.14	Apport Personnel et Professionnel	34
2.15	Mot de la Fin	34
Bibliographie		36

Table des figures

1.1	Interface de ruiji cloud	4
1.2	Interface de Cisco Meraki	5
1.3	Interface de Ubiquiti UniFi	6
1.4	Interface de zabbix	7
1.5	processus Scrum.	10
1.6	Les acteurs de scrum.	11
2.1	Diagramme des cas d'utilisation global	23
2.2	Diagramme de classes global	25
2.3	Notre Architecture proposée pour la plateforme.	28

Liste des tableaux

1.1	analyse comparative.	8
-----	------------------------------	---

Introduction Générale

Dans un environnement technologique en constante mutation, la fiabilité des infrastructures réseau constitue le pilier fondamental de la transformation numérique et de la continuité des services en entreprise. Face à la multiplication des équipements et à la complexité croissante des architectures, la gestion des solutions de connectivité nécessite désormais des outils de pilotage centralisés à la hauteur des enjeux de performance actuels.

Ce projet s'inscrit précisément dans cette dynamique en proposant la conception et le développement d'une plateforme web unifiée dédiée à la gestion et à la supervision des réseaux Ruijie. L'objectif principal est de pallier les limites d'une gestion actuellement fragmentée, qui augmente les risques d'erreurs et complexifie le suivi opérationnel au quotidien. En regroupant l'intégralité des projets au sein d'une interface unique, la plateforme optimise les performances et simplifie la maintenance grâce à une visualisation graphique de la topologie et un suivi en temps réel du statut ainsi que des performances des équipements.

L'innovation majeure de cette solution réside dans l'intégration d'un moteur d'intelligence artificielle conçu pour analyser les comportements anormaux, anticiper les défaillances potentielles et suggérer des optimisations proactives du réseau. Pour répondre aux besoins stratégiques et communicationnels de l'organisation, l'écosystème intègre également un système d'envoi automatisé pour les alertes critiques ainsi qu'un module de communication interne dédié aux annonces importantes. Enfin, la sécurité du système est garantie par une gestion rigoureuse des accès basée sur les rôles, assurant ainsi l'intégrité et la confidentialité des données stockées.

1

Présentation générale du projet

1.1 Introduction

Ce premier chapitre s'articule autour de trois axes principaux visant à poser les fondements de notre travail. Dans un premier temps, nous présentons la société d'accueil au sein de laquelle ce projet de fin d'études a été réalisé. Par la suite, nous exposons une analyse critique de la situation actuelle, mettant en lumière les limites de la gestion dispersée des réseaux Ruijie, afin d'introduire la solution centralisée que nous proposons pour optimiser la supervision et le suivi opérationnel. Enfin, nous concluons cette partie en détaillant la méthodologie de conception adoptée pour structurer le développement de la plateforme, de l'analyse des besoins jusqu'à la mise en œuvre technique

1.2 Organisme d'accueil

1.2.1 Présentation de société

Ce stage de fin d'études s'est déroulé chez "**SOTUPUB**", une société spécialisée dans les services informatiques, la sécurisation et le développement sur mesure. L'expertise de l'entreprise en installation et maintenance d'infrastructures offre le cadre idéal pour le développement de notre plateforme de supervision centralisée Ruijie. Ce projet vient ainsi renforcer l'offre de **SOTUPUB** en optimisant le suivi opérationnel et la performance des réseaux de ses clients.



1.2.2 Les Services

SOTUPUB se concentre sur huit domaines d'expertise clés :

1. Fourniture et installation de systèmes informatiques.
2. Développement d'applications web et mobiles.
3. Maintenance et sécurisation des systèmes.
4. Contrats de maintenance et infogérance.
5. Suivi de parc informatique.
6. Vente de matériel informatique.
7. Accompagnement au développement informatique.
8. Technologie de pointe.

1.3 Etude préalable

1.3.1 Contexte de projet

Dans le cadre de la gestion des projets réseau Ruijie, l'entreprise fait face à une dispersion des outils de suivi qui complexifie la supervision et augmente les risques d'erreurs opérationnelles. Pour remédier à cette situation, ce projet propose la création d'une interface web unique conçue pour centraliser l'ensemble des infrastructures réseau et optimiser leurs performances globales.

La plateforme offre une visibilité complète grâce à une visualisation graphique de la topologie et un suivi en temps réel du statut des équipements. Au-delà de la simple supervision, elle intègre un moteur d'intelligence artificielle pour l'analyse des comportements et la détection d'anomalies, garantissant ainsi une maintenance proactive. Enfin, pour fluidifier la communication et la réactivité, le système centralise les alertes par mailing et intègre un module d'annonces internes pour une coordination optimale des équipes.

1.3.2 Problématique

Actuellement, la gestion et le suivi des projets réseau Ruijie sont fragmentés et dispersés, ce qui complexifie considérablement la supervision opérationnelle. Cette absence de centralisation entraîne

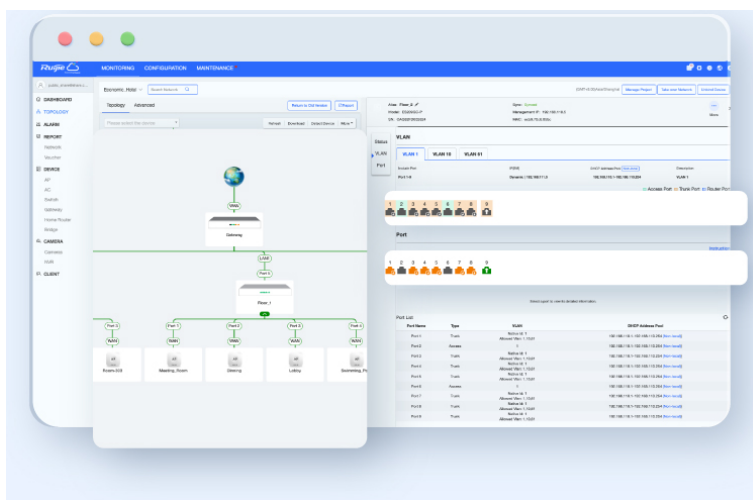
une augmentation significative des risques d'erreurs de configuration et nuit à la détection des incidents en temps réel. L'enjeu est donc de remédier à ce manque de visibilité globale en concevant une solution capable de fédérer l'ensemble des infrastructures au sein d'une interface unique pour optimiser les performances et la maintenance.

1.3.3 Étude de l'existant

À ce jour, un certain nombre d'acteurs clés contrôlent le marché des plateformes professionnelles et collaboratives, chacune répondant à un besoin spécifique. Cependant, ces solutions restent souvent fragmentées et ne fournissent pas une expérience holistique aux utilisateurs.

1.3.3.1 ruiji cloud [1]

Ruijie Cloud est la plateforme officielle de gestion centralisée basée sur le Cloud, conçue par le constructeur Ruijie Networks. Elle permet aux administrateurs réseau de configurer, gérer et surveiller leurs équipements (Points d'accès, Switches, Gateways) à distance.



: Interface de ruiji cloud

1.3.3.1.1 Avantages :

- **Facilité** : Installation rapide des équipements (Plug & Play).
- **Gratuité** : Une version gratuite performante pour la gestion de base.
- **Mobilité** : Disponible sur le web et sur application mobile.

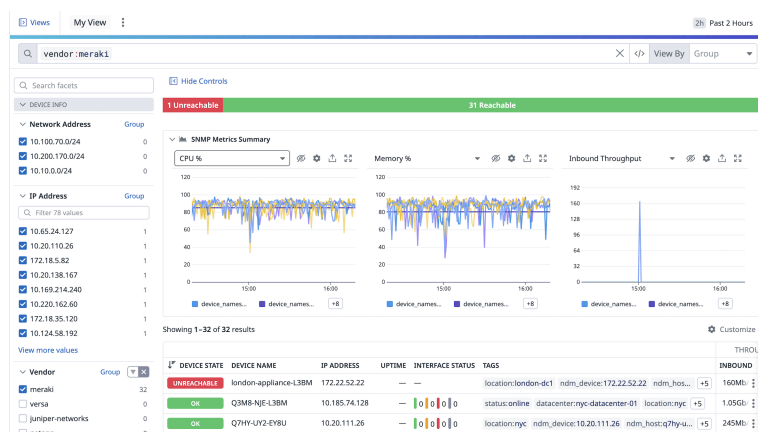
1.3.3.1.2 Inconvénients :

- **Manque de fonctions** : Impossible d'y intégrer des publicités ou une messagerie interne.

- **Interface figée** : On ne peut pas personnaliser l'outil selon les besoins de l'entreprise.
- **IA limitée** : Elle donne des rapports simples mais n'anticipe pas intelligemment les pannes.

1.3.3.2 Cisco Meraki [2]

Cisco Meraki est le leader mondial des solutions réseau gérées via le Cloud. Elle propose une gestion centralisée de bout en bout (Switching, Wi-Fi, Sécurité, SD-WAN) à travers un tableau de bord unique appelé le "Meraki Dashboard". C'est la solution qui a popularisé le concept de supervision holistique.



: Interface de Cisco Meraki

1.3.3.2.1 Avantages :

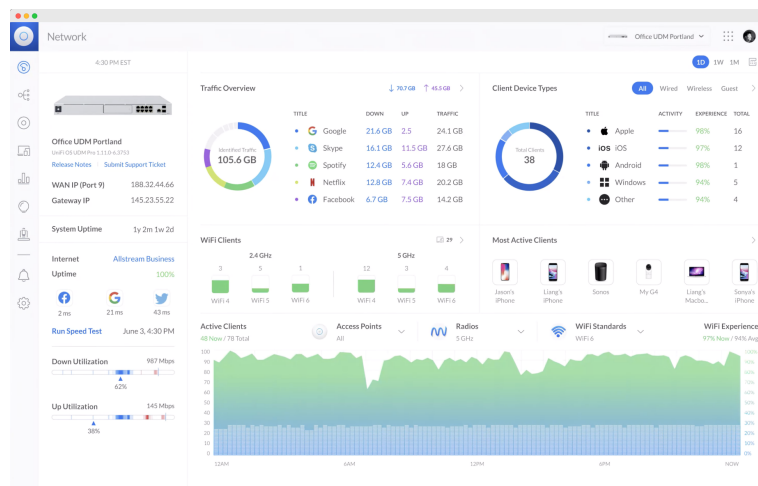
- **Simplicité** : Une interface très facile à utiliser et très visuelle.
- **Visibilité** : Analyse précise de tout ce qui se passe sur le réseau.
- **Sécurité** : Mise à jour automatique des équipements.

1.3.3.2.2 Inconvénients :

- **Prix** : Solution très coûteuse (licences obligatoires).
- **Rigidité** : Impossible d'ajouter des fonctions personnalisées (comme vos publicités internes).
- **Dépendance** : Si la licence expire, le matériel s'arrête de fonctionner.

1.3.3.3 Ubiquiti UniFi [3]

C'est une solution très utilisée par les PME pour gérer le Wi-Fi et les équipements réseau via une interface moderne et visuelle.



: Interface de Ubiquiti UniFi

1.3.3.3.1 Avantages :

- **Zéro coût** : Le logiciel de gestion est gratuit (pas de licence).
- **Topologie visuelle** : Offre de très beaux graphiques pour voir comment les appareils sont connectés.
- **Simplicité** : Très facile à installer et à agrandir.

1.3.3.3.2 Inconvénients :

- **Support limité** : En cas de problème, il faut souvent chercher la solution soi-même.
- **Pas d'IA avancée** : L'outil surveille le réseau mais n'anticipe pas intelligemment les pannes.
- **Outil technique uniquement** : Il ne permet pas de gérer la communication interne ou les publicités.

1.3.3.4 Zabbix / Grafana [4]

ce sont des outils de surveillance (Monitoring) très puissants. **Zabbix** s'occupe de collecter les données techniques des machines, tandis que **Grafana** les transforme en tableaux de bord (Dashboards) visuels et modernes.



: Interface de zabbix

1.3.3.4.1 Avantages :

- **Personnalisation** : On peut créer des graphiques très précis et détaillés.
- **Puissance** : Ils peuvent surveiller presque tout (serveurs, réseaux, bases de données).
- **Open Source** : Les logiciels sont gratuits et disposent d'une grande communauté.

1.3.3.4.2 Inconvénients :

- **Grande Complexité** : Ils sont très difficiles à installer et à configurer pour un débutant.
- **Uniquement de l'observation** : Ils permettent de "voir" les pannes, mais pas de "gérer" ou de "configurer" les équipements Ruijie.
- **Interface austère** : L'interface de Zabbix est très technique et n'est pas adaptée à des utilisateurs non-experts.

1.3.4 Etude comparative

Bien que les outils du marché offrent des performances réseaux solides, ils restent des solutions génériques et fermées. L'impossibilité d'y intégrer des modules personnalisés, tels que la communication interne ou une analyse prédictive avancée, justifie pleinement le développement de notre plateforme. Ce projet vise à offrir une solution sur mesure, capable de fusionner la gestion technique et les besoins opérationnels spécifiques de l'entreprise.

Critères de comparaison	Notre Sol.	Ruijie	Cisco	UniFi	Zabbix
Prix d'acquisition (Logiciel)	✓	✓	X	✓	✓
Frais de licence annuels	✓	✓	X	✓	✓
Collecte de données automatique	✓	✓	✓	✓	X
Gestion de Publicité (Marketing)	✓	X	X	X	X
Coût de maintenance IA	✓	X	X	X	X
Rentabilité (Marketing intégré)	✓	X	X	X	X
Indépendance financière (APIs)	X	✓	X	✓	✓

: analyse comparative.

Légende :

- Vert clair : Avantage
- Rouge clair : Limitation majeure.
- ✓ : Fonctionnalité disponible
- X : Fonctionnalité non disponible

Les données sont basées sur les analyse de marché Gartner et G2.

1.3.5 Solution proposée

Notre projet vise à créer un outil de gestion réseau intelligent qui fait bien plus que surveiller des machines. Cette plateforme permet de contrôler tous les équipements à distance depuis une seule interface simple. Sa grande force est l'utilisation de l'**intelligence artificielle** : elle analyse le réseau pour deviner les pannes avant qu'elles n'arrivent, ce qui évite les coupures de connexion. En plus de l'aspect technique, nous avons ajouté des outils de **marketing**, comme l'envoi d'e-mails automatiques et l'affichage de publicités. Cela permet de transformer le réseau internet de l'entreprise en un véritable moyen de communication. En résumé, c'est une solution tout-en-un qui aide à la fois les techniciens à garder un réseau stable et les responsables à mieux communiquer avec leurs clients.

1.4 Méthodologie de travail

La réalisation de ce projet, à la convergence du développement Web et de l'intégration de modules d'Intelligence Artificielle, exige une structure méthodologique alliant rigueur conceptuelle et flexibilité opérationnelle.

Évoluant en autonomie sur l'ensemble du cycle de vie du logiciel (SDLC), mon défi principal

réside dans la synchronisation entre l'ingénierie des données et l'implémentation d'interfaces ergonomiques répondant aux standards actuels. Pour pallier les risques liés à la complexité technique et assurer une livraison incrémentale de haute qualité, j'ai adopté la méthodologie Agile. Ce choix stratégique favorise une gestion proactive des interdépendances technologiques et une visibilité accrue sur l'état d'avancement des travaux.

Dans cette optique, j'ai implémenté le cadre Scrum. Bien que traditionnellement collaboratif, l'usage individuel du Scrum m'offre une discipline rigoureuse à travers la planification de Sprints thématiques. Cette approche itérative garantit non seulement une réactivité face aux imprévus techniques, mais assure également une amélioration continue des fonctionnalités, de la phase de prototypage jusqu'au déploiement final.

1.4.1 Framework Scrum

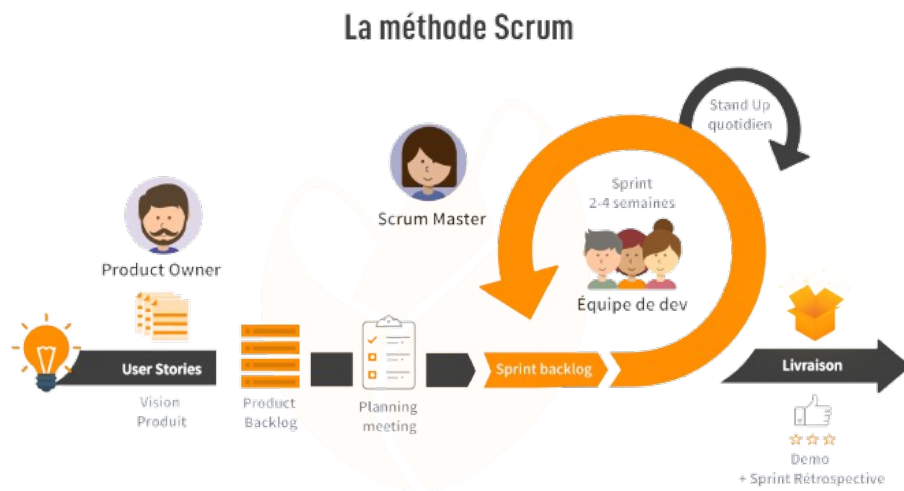
Scrum [5] : un framework empirique permettant de naviguer dans la complexité technique tout en garantissant une livraison de produits à haute valeur ajoutée. Loin d'être une méthode rigide ou un processus linéaire, Scrum offre un environnement structuré qui favorise l'agilité intellectuelle et la créativité opérationnelle.

En adoptant ce cadre de travail, l'accent est mis sur la transparence et l'inspection régulière, permettant ainsi d'évaluer l'efficacité des stratégies de développement mises en œuvre. Pour mon projet, l'application de Scrum facilite une amélioration continue non seulement des fonctionnalités techniques, mais aussi de l'organisation personnelle. Il permet d'ajuster les techniques de conception de manière itérative, assurant ainsi une adéquation constante entre les besoins technologiques et les solutions développées.

1.4.2 Processus Scrum

Scrum s'impose comme un cadre de travail Agile structurant la gestion de projet autour d'un processus itératif et incrémental. En découpant le développement en cycles courts, nommés sprints (généralement de 2 à 4 semaines), il garantit la livraison régulière d'un produit fonctionnel. Cette dynamique repose sur le triptyque de l'empirisme : la transparence, qui offre une visibilité totale sur l'avancement à l'ensemble des acteurs ; l'inspection, qui permet d'évaluer périodiquement les résultats et les méthodes pour détecter tout écart ; et l'adaptation, qui offre la flexibilité nécessaire pour ajuster la trajectoire selon les retours des utilisateurs. Le déroulement précis de ce flux opérationnel est détaillé dans la figure 1.5, illustrant la transition du besoin initial vers la valeur ajoutée finale.

La figure 1.5 illustre le processus de déroulement de développement Scrum :

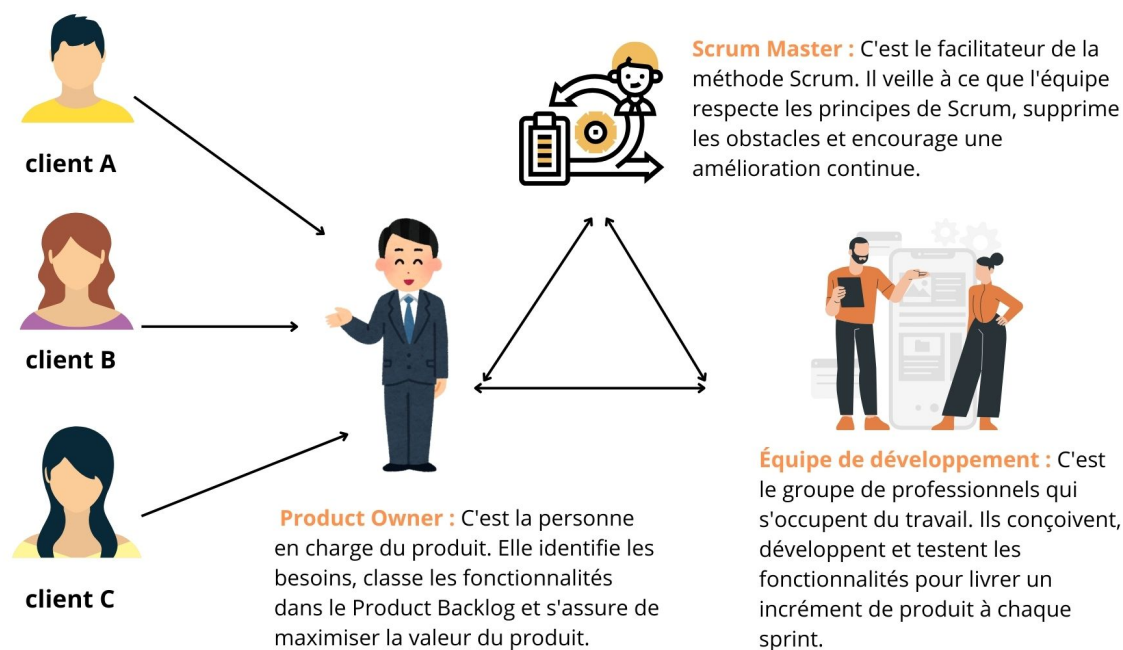


: processus Scrum.

1.4.3 Les avantages de Scrum

L'un des atouts majeurs de Scrum réside dans sa grande flexibilité, permettant aux équipes de s'adapter avec agilité aux évolutions du marché et aux nouvelles exigences grâce à son approche itérative. Le rythme soutenu des sprints (2 à 4 semaines) garantit une livraison continue de valeur, favorisant ainsi un cycle de retours utilisateurs rapide et constructif. Ce cadre renforce également la transparence et la cohésion d'équipe par le biais de rituels quotidiens et de revues de fin de cycle, assurant un alignement constant sur les objectifs du projet. Enfin, Scrum instaure une culture d'amélioration continue : chaque rétrospective devient un levier pour optimiser les processus internes, lever les obstacles et accroître durablement la productivité collective.

1.4.4 Les acteurs de scrum



: Les acteurs de scrum.

1.4.5 Les artefacts de scrum

1. Product Backlog (Backlog Produit)

Le Product Backlog constitue l'unique source d'exigences pour le produit. Il s'agit d'une liste dynamique et ordonnée par priorité, recensant l'ensemble des fonctionnalités, correctifs et évolutions envisagés. Sous la responsabilité du Product Owner, ce registre est continuellement affiné et mis à jour pour refléter les besoins changeants des parties prenantes et garantir la pertinence de la valeur livrée.

2. Sprint Backlog (Backlog du Sprint)

Le Sprint Backlog regroupe les éléments prioritaires extraits du Product Backlog que l'équipe s'engage à réaliser durant le cycle en cours. Il contient non seulement les fonctionnalités à développer, mais aussi un plan d'action concret, souvent segmenté en tâches techniques précises. Exclusivement géré et actualisé par l'équipe de développement, cet artefact offre une visibilité en temps réel sur l'effort nécessaire pour atteindre l'objectif du sprint.

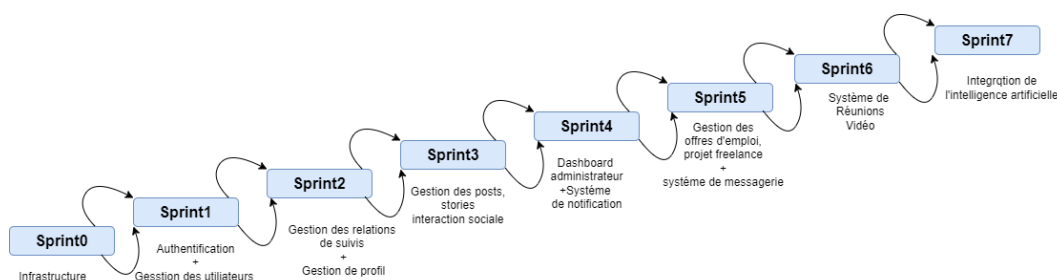
3. Increment (Incrément)

L'Incrément représente la valeur concrète ajoutée au produit à l'issue de chaque sprint. Il

s'agit d'une version fonctionnelle, testée et potentiellement livrable, qui matérialise l'avancement global du projet. Pour être validé, chaque incrément doit impérativement satisfaire à la Définition de Terminé (Definition of Done), un référentiel de qualité strict qui garantit que chaque fonctionnalité répond aux standards techniques et métier exigés.

1.5 Planification itérative du projet

Pour garantir une organisation optimale de nos travaux, nous avons adopté une démarche Agile en segmentant le projet en plusieurs sprints. Chaque itération constitue une étape jalonnée par des objectifs spécifiques à réaliser dans un intervalle de temps défini. Ce découpage stratégique facilite un suivi rigoureux de l'état d'avancement, une gestion agile des priorités et une réactivité accrue face aux ajustements nécessaires. La répartition détaillée des tâches par sprint est présentée dans la figure ci-après.



1.5.1 Langage de modélisation UML

La phase de conception de notre application s'appuie sur le langage de modélisation UML (Unified Modeling Language). L'adoption de ce standard nous a permis d'élaborer une représentation visuelle cohérente et rigoureuse du système. À travers divers diagrammes, nous avons pu modéliser avec précision la structure des données, la logique comportementale des objets ainsi que les flux d'interactions entre les différents composants logiciels.[6]

1.6 Conclusion

Ce premier chapitre a posé les bases de notre étude en exposant le contexte, les objectifs majeurs et les exigences spécifiques de notre projet, tout en analysant l'existant. Nous y avons également justifié l'adoption du framework Scrum comme méthodologie de gestion, en soulignant la valeur ajoutée de son approche itérative. Enfin, l'introduction du langage de modélisation UML a permis de définir le cadre conceptuel nécessaire à la conception de l'application. Les chapitres suivants seront

consacrés au déploiement technique de cette solution, de sa conception détaillée à sa réalisation effective.

2

Architecture, outils et environnement de développement

2.1 Introduction

Ce chapitre présente le sprint zéro qui représente le premier pas de réalisation de notre projet. Tout d'abord, nous identifions les acteurs de notre application. Puis, nous listons les besoins fonctionnelles et non fonctionnelles de notre système et nous détaillons le travail par la méthodologie choisie dans le chapitre précédent. Enfin, nous donnons un bref aperçu sur le matériel de base, les technologies et les langages de programmation utilisés pour la mise en place de l'environnement de travail.

2.2 Capture des besoins

2.2.1 Identification des acteurs

L'application se compose de trois types d'acteurs principaux : Administrateur, Manager et Système IA. Chaque acteur dispose de responsabilités spécifiques en fonction de son rôle dans la plateforme.

Administrateur

L'administrateur est l'acteur central en charge de la gestion globale de la plateforme. Il dispose d'un contrôle total sur les utilisateurs, les équipements réseau, les projets et les statistiques.

Ses responsabilités incluent la gestion des comptes utilisateurs et des rôles, la supervision de l'ensemble des projets réseau Ruijie, la gestion des publicités internes, ainsi que la consultation des rapports et des statistiques de la plateforme.

Manager

Le manager est l'acteur responsable du suivi opérationnel des projets et des clients.

Il peut consulter et gérer les projets réseau qui lui sont assignés, visualiser la topologie réseau, superviser en temps réel l'état des équipements, accéder aux alertes et notifications, et générer des rapports techniques et statistiques en PDF ou CSV.

Système IA

Le système IA est un acteur automatisé qui interagit avec la plateforme via une API REST.

Il analyse en continu les données de supervision réseau afin de détecter les comportements anormaux, anticiper les défaillances potentielles et proposer des actions d'optimisation.

Ses résultats sont transmis au backend et affichés en temps réel sur le tableau de bord.

2.2.2 Besoins fonctionnels

Les besoins fonctionnels du système décrivent les actions attendues de chaque acteur dans la plateforme. Ces fonctionnalités répondent aux exigences en matière de supervision, de gestion et d'analyse réseau.

Pour l'Administrateur

- Gérer les comptes utilisateurs (créer, consulter, modifier, supprimer) et attribuer les rôles.
- Gérer l'ensemble des projets réseau Ruijie et des clients associés.
- Modérer et gérer les publicités et annonces internes diffusées sur la plateforme.
- Consulter les statistiques globales de la plateforme (activité des équipements, répartition des projets, historique des alertes).
- Générer et exporter des rapports techniques et statistiques en PDF et CSV.

Pour le Manager

- Consulter et suivre les projets réseau et les clients rattachés à son périmètre.
- Visualiser la topologie et l'architecture réseau de chaque projet.
- Superviser en temps réel l'état, le trafic et les ressources des équipements réseau.

- Recevoir et consulter les alertes et notifications critiques via le système de mailing.
- Configurer de manière sécurisée les paramètres réseau standard des équipements.
- Générer des rapports de supervision et d'analyse pour les projets dont il est responsable.
- Accéder au tableau de bord et à l'historique des incidents et des logs

Pour le Système IA

- Analyser en continu les données de trafic et de performance des équipements réseau.
- Détecter automatiquement les comportements anormaux et les anomalies réseau.
- Anticiper les défaillances potentielles avant qu'elles n'impactent les projets.
- Proposer des suggestions d'optimisation des configurations réseau.
- Communiquer les résultats d'analyse au backend via API REST pour affichage en temps réel sur la plateforme.

2.2.3 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les critères qualitatifs importants pour garantir une expérience utilisateur fluide et un fonctionnement optimal de l'application.

- **Fiabilité** : Le backend sous NestJS doit offrir une stabilité optimale pour la supervision continue. Le système de mailing doit garantir la réception systématique des alertes critiques sans perte d'information.
- **Sécurité** : Les accès sont protégés par le protocole HTTPS et l'authentification JWT. Le contrôle d'accès basé sur les rôles (RBAC) assure la confidentialité des données entre les administrateurs, managers et techniciens.
- **Scalabilité** : L'architecture s'appuyant sur MongoDB permet de gérer une augmentation fluide du volume de données liées aux nouveaux projets et équipements. La structure modulaire facilite l'extension vers de nouveaux services sans saturer le système.
- **Maintenabilité** : Le code doit être structuré et documenté pour permettre des mises à jour aisées du module IA et des fonctionnalités de gestion.
- **Compatibilité** : La plateforme doit être responsive et multi-plateforme pour permettre aux techniciens d'accéder à la topologie réseau depuis n'importe quel terminal.

2.3 Product Backlog

En tant que Product Owner, je traduis les besoins documentés en un Product Backlog structuré, où chaque fonctionnalité est rédigée sous forme de User Stories (ex : gestion des accès , topologie

graphique et analyse IA) pour garantir la valeur métier. Une étape cruciale consiste à appliquer une technique de priorisation rigoureuse, hiérarchisant les éléments selon leur importance stratégique : la supervision en temps réel et la sécurité (JWT/RBAC) sont classées en priorité Haute , tandis que la génération de rapports et le système de publicité interne sont classés en priorité Moyenne.

Groupe de Fonctionnalités	User Stories	Priorité
Authentification & Gestion des Utilisateurs	– En tant qu'utilisateur, Je peux créer un compte en renseignant mes informations professionnelles.	Haute
	– En tant qu'utilisateur, Je peux choisir mon type de profil (utilisateur standard ou entreprise) lors de l'inscription.	Haute
	– En tant qu'utilisateur, je peux me connecter à mon compte en toute sécurité.	Haute
	– En tant qu'administrateur, Je peux gérer les utilisateurs : voir la liste, consulter les détails, supprimer un compte, etc.	Haute
Gestion de Profil & Gestion des relations de suivi entre les utilisateurs	– En tant qu'utilisateur, Je peux gérer les informations de mon profil professionnel afin de refléter mon identité et mon parcours.	Haute
	– En tant qu'utilisateur, Je peux valoriser mon parcours professionnel et académique à travers les éléments présentés dans mon profil.	Haute
	– En tant qu'utilisateur, Je peux mettre en avant mes compétences pour mieux représenter mes expertises.	Moyenne
	– En tant qu'utilisateur, Je peux consulter les profils des autres utilisateurs afin de découvrir leur parcours professionnel.	Moyenne
	– Je peux modifier et mettre à jour les informations de mon profil pour qu'elles restent pertinentes.	Haute

	– En tant qu'utilisateur, Je peux suivre d'autres utilisateurs afin de rester informé de leur activité et de leur évolution professionnelle.	Moyenne
Publication de post	– En tant qu'utilisateur, Je peux publier et gérer du contenu professionnel. – En tant qu'utilisateur, Je peux enrichir mes publications (multimédia, liens, hashtags, tags). – En tant qu'utilisateur, Je peux accéder et rechercher des publications pertinentes, notamment celles des abonnements.	Haute Moyenne Moyenne
Interactions Sociales	– En tant qu'utilisateur, Je peux interagir avec les publications (aimer, commenter, répondre aux commentaires). – En tant qu'utilisateur, Je peux gérer mes propres commentaires (modifier, supprimer). – En tant qu'utilisateur, Je peux visualiser les statistiques d'interactions (nombre de likes, commentaires). – En tant qu'utilisateur, Je peux identifier qui a aimé mes publications.	Moyenne Moyenne Basse Basse
Stories	– En tant qu'utilisateur, Je peux créer, personnaliser et gérer des stories éphémères (texte, images, vidéos). – En tant qu'utilisateur, Je peux consulter les stories des utilisateurs que je suis. – En tant qu'utilisateur, Je peux faire expirer automatiquement mes stories après 24 heures. – En tant qu'utilisateur, Je peux voir qui a vu mes stories.	Moyenne Moyenne Moyenne Basse
Dashboard	– En tant qu'administrateur, Je peux consulter un tableau de bord avec des statistiques globales sur l'activité et l'engagement des utilisateurs et des contenus.	Haute

Notifications	– En tant qu'utilisateur, Je peux recevoir et gérer mes notifications en temps réel pour rester informé des interactions importantes. – En tant qu'utilisateur, Je peux organiser mes notifications facilement (marquer comme lues, supprimer).	Haute Moyenne
Projets Freelance	– En tant qu'employeur, Je peux publier et gérer mes projets (description, budget, durée, compétences, documents) – En tant que freelance, Je peux rechercher des projets et soumettre des propositions adaptées (lettre de motivation, prix, durée) – En tant qu'utilisateur, Je peux gérer le cycle contractuel (génération, personnalisation, signature électronique) – En tant qu'utilisateur, Je peux recevoir des notifications sur l'avancement des projets et les échéances contractuelles	Haute Haute Haute Moyenne
Messagerie	– En tant qu'utilisateur, Je peux envoyer et recevoir des messages directs et de groupe pour communiquer avec d'autres utilisateurs. – Je peux partager des médias (images, fichiers, vidéos) dans mes conversations. – Je peux recevoir des notifications en temps réel pour les nouveaux messages afin de rester informé. – En tant qu'utilisateur, Je peux rechercher dans l'historique de mes conversations et voir le statut de lecture des messages. – En tant qu'utilisateur, Je peux gérer les membres d'une conversation de groupe (ajouter, retirer, quitter) pour organiser mes discussions.	Haute Moyenne Haute Basse Moyenne

Offres d'Emploi	– En tant qu'entreprise, Je peux publier et gérer des offres d'emploi complètes et localisées. – En tant qu'entreprise, Je peux consulter, gérer les candidatures et suivre leur statut. – En tant qu'utilisateur, Je peux rechercher et filtrer des offres d'emploi selon plusieurs critères (localisation, domaine, contrat). – En tant qu'utilisateur, Je peux consulter, signer électroniquement et accepter un contrat d'embauche. – En tant qu'entreprise, Je peux générer automatiquement un contrat lors de l'acceptation d'un candidat. – En tant qu'entreprise, Je peux activer ou désactiver mes offres selon mes besoins.	Haute Moyenne Haute Haute Haute Moyenne
Système de Réunions	– En tant qu'utilisateur, Je peux organiser et lancer des réunions vidéo (titre, description, durée, type) – En tant qu'utilisateur, Je peux inviter, rejoindre et terminer des réunions pour collaborer efficacement – En tant qu'utilisateur, Je peux gérer les enregistrements et supports (activables, présentations) – En tant qu'utilisateur, Je peux consulter l'historique et les statistiques de mes réunions (participants, durée, accès aux enregistrements) – En tant qu'utilisateur, Je peux recevoir des notifications et rappels pour mes réunions planifiées et invitations reçues	Haute Haute Moyenne Moyenne Haute

Entretien (Intelligence Artificielle)	– Je peux générer et visualiser des questions d’entretien personnalisées selon mon domaine et le type de poste, et recevoir une évaluation détaillée de mes réponses.	Haute
	– En tant qu’utilisateur, Je peux soumettre mon CV pour identifier les offres les plus pertinentes et obtenir un score de correspondance avec des explications.	Haute
	– En tant qu’utilisateur, Je peux gérer la base de connaissances (ajout de documents) pour alimenter la génération et l’évaluation des questions.	Moyenne
	– En tant qu’utilisateur, Je peux trouver et évaluer les candidats les plus adaptés à une offre grâce à un système de score et de feedback détaillé.	Haute

Le Product Backlog constitue un pilier fondamental de la méthodologie agile adoptée pour ce projet. Il regroupe, sous forme d’une liste priorisée, l’ensemble des fonctionnalités, des améliorations et des corrections à apporter à l’application, appelées "user stories". Chaque user story décrit une fonctionnalité précise, inclut les critères d’acceptation ainsi qu’une estimation de l’effort requis pour sa réalisation. Le Product Backlog est un document évolutif, régulièrement mis à jour et ajusté tout au long du projet afin d’assurer que l’équipe se concentre en permanence sur les tâches les plus prioritaires et les plus pertinentes pour atteindre les objectifs fixés.

2.4 Les Diagrammes

2.4.1 Diagramme de cas d’utilisation global

Le diagramme de cas d’utilisation est un outil essentiel permettant de visualiser les interactions entre les différents acteurs et le système. Il offre une vue d’ensemble des actions que peuvent effectuer les utilisateurs tels que l’entreprise, utilisateurs et l’administrateur. Ce diagramme met en évidence les principales fonctionnalités du système et illustre la manière dont chaque acteur les utilise. Il facilite non seulement la compréhension des échanges entre le système et ses utilisateurs, mais

permet également de détecter d'éventuelles insuffisances ou incohérences dans le fonctionnement global de l'application.



: Diagramme des cas d'utilisation global .

2.4.2 Diagramme de classes global

Le diagramme de classe est un autre élément important de notre documentation de conception. Il décrit la structure de notre système en montrant les classes, leurs attributs et méthodes, ainsi que les relations entre elles. Il sert de plan pour la mise en œuvre du code, aidant à comprendre comment les différentes parties du système interagissent entre elles. Le diagramme de classe aide à assurer que notre conception est solide, logique et bien organisée, ce qui facilitera la phase de développement.

2.5 Architecture de l'application

Dans notre projet, nous avons adopté une architecture microservices distribuée, pensée pour répondre aux besoins de modularité, de scalabilité et de résilience. Chaque microservice est dédié à une fonctionnalité métier spécifique (authentification, gestion des publications, notifications, profils, etc.), ce qui permet une meilleure organisation du code, une isolation des erreurs et une évolution indépendante des différentes briques de l'application. À l'intérieur de chaque microservice, nous avons adopté le modèle d'architecture **MVC (Modèle–Vue–Contrôleur)**, ce qui garantit une séparation claire des responsabilités entre la logique métier, la présentation et les interactions utilisateur, facilitant ainsi la maintenance et les tests unitaires.

L'entrée principale des requêtes passe par un **API Gateway**, qui joue un rôle d'orchestrateur central. Il redirige les requêtes vers les microservices concernés, applique les règles de sécurité via JWT, gère le routing, le filtrage, ainsi que la limitation de débit. Grâce à un **Service Discovery**, chaque microservice peut s'enregistrer dynamiquement et découvrir les autres services sans dépendance directe aux adresses IP, renforçant ainsi la tolérance aux pannes et la flexibilité du déploiement.

Afin de garantir une répartition optimale de la charge réseau, notre backend est également placé derrière un **load balancer**. Ce composant est essentiel pour la scalabilité horizontale : il permet de distribuer les requêtes de manière équilibrée entre plusieurs instances de microservices, améliorant les performances et la disponibilité du système. Cela garantit aussi que les utilisateurs ne subissent pas de ralentissements, même en cas de forte affluence.

Chaque service est conteneurisé avec **Docker** et orchestré à l'aide d'un fichier `docker-compose.yml` qui configure les variables d'environnement nécessaires à la communication inter-services (ex : `POST_SERVICE_URL`, `AUTH_SERVICE_URL`, etc.), tout en connectant les conteneurs via un réseau privé Docker (`backend_network`). Conformément aux bonnes pratiques, chaque service possède sa propre base de données pour garantir l'indépendance et l'autonomie de chaque composant.

Côté stockage de données, nous utilisons **MongoDB** pour les services manipulant des structures souples et évolutives (publications, profils, stories, etc.), et **PostgreSQL** avec l'ORM **Sequelize** pour les services nécessitant une structure relationnelle stricte, comme l'authentification ou les notifications.

En ce qui concerne la sécurité, tous les services valident les **JSON Web Tokens (JWT)** pour garantir l'authentification et l'autorisation des utilisateurs. Cela permet de centraliser la gestion des identités tout en conservant une architecture distribuée et sécurisée.

Pour la communication en temps réel, notre projet combine **WebSockets**, qui permettent une

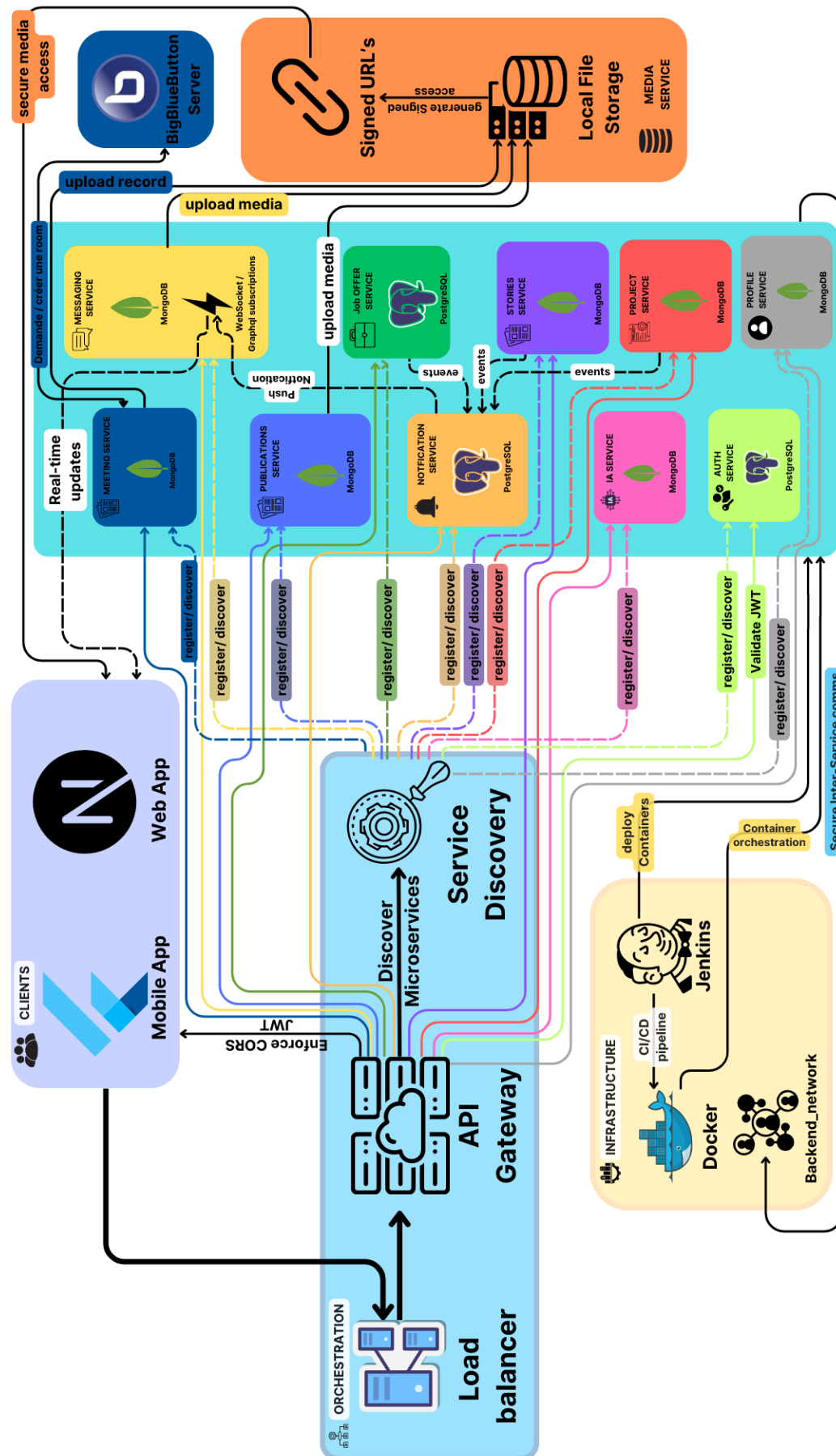
transmission instantanée des messages et notifications, et **GraphQL Subscriptions**, qui assurent une synchronisation dynamique des interfaces utilisateurs avec les données serveur, notamment lors de changements d'état.

Le stockage de fichiers est géré localement lors de l'upload initial, puis les médias sont accessibles via des **URLs signées**, offrant un accès sécurisé et temporaire aux ressources partagées (images, vidéos, etc.).

Afin d'intégrer des fonctionnalités intelligentes, nous avons ajouté un **microservice IA** reposant sur une architecture **RAG (Retrieval-Augmented Generation)**, développé avec **Flask** et le modèle **Mistral 7B**. Ce service est dédié à l'assistance utilisateur, à la recherche contextuelle, à la génération de réponses personnalisées ou à l'analyse de contenu. Il s'appuie sur une base vectorielle pour effectuer la recherche sémantique, puis utilise Mistral 7B pour générer des réponses en langage naturel. Il respecte également la structure **MVC** et s'intègre au système via l'API Gateway, en respectant les protocoles de sécurité (JWT). Cela permet une utilisation fluide par le front-end mobile et web, tout en conservant la robustesse de l'architecture distribuée.

Pour assurer la qualité continue du code et automatiser les déploiements, nous avons intégré **Jenkins** dans notre chaîne DevOps. Jenkins est configuré pour effectuer des builds automatisés, exécuter des tests, vérifier les bonnes pratiques de code et déployer automatiquement les nouveaux conteneurs Docker après validation. Cette approche favorise une intégration et un déploiement continus (**CI/CD**), réduisant ainsi le temps entre l'écriture du code et sa mise en production, tout en garantissant une plus grande fiabilité.

Enfin, le front-end de notre projet est assuré par deux interfaces : une application mobile développée avec **Flutter**, qui garantit une expérience fluide sur Android et iOS, et une application web développée avec **Next.js**, qui bénéficie du rendu côté serveur (**Server-Side Rendering**) pour un chargement rapide et une meilleure indexation SEO. Ces deux interfaces consomment les API exposées par les microservices uniquement via l'**API Gateway**, assurant ainsi une couche d'abstraction propre et sécurisée entre le client et le backend.



: Notre Architecture proposée pour la plateforme.

2.6 Outils et environnement de développement

2.6.1 Environnement matériel

Dans le cadre de ce projet, plusieurs machines ont été utilisées pour le développement et les tests. Voici les caractéristiques principales des équipements utilisés :

- PC 1 (GIGABYTE) :
 - Carte graphique : NVIDIA GeForce RTX 3060
 - Mémoire RAM : 16 Go
 - Disque : SSD NVMe de 512 Go
- **PC 2 (Lenovo)** :
 - Mémoire RAM : 8 Go
 - Disque : SSD NVMe de 512 Go

2.6.2 Environnement technique

	Next.js :[7] Framework React pour le rendu SSR optimisé et la performance via le pré-rendu statique.
	API Gateway :[8] Centralise les appels vers les microservices, applique les règles de sécurité (JWT), et gère les erreurs/quota.
	Flask : [9] Micro-framework Python léger pour concevoir des APIs RESTful et interfaces web d'administration.
	Mistral 7B & Ollama : [10] [11] Utilisés pour des fonctionnalités IA (résumé, génération, analyse). Ollama permet l'exécution locale du modèle Mistral 7B.
	Gestion de Version (GitLab & GitHub) : [12] [13] Nous avons utilisé GitLab pour la gestion de version de notre code source et pour les pipelines CI/CD. GitHub a également été utilisé pour héberger certains dépôts open-source.
	Visual Studio Code : [14] Éditeur de code moderne avec des extensions utiles au développement web, mobile, IA et DevOps.

	Docker & Docker Compose : [15] Conteneurisation des services avec Docker. Orchestration facilitée par Docker Compose pour le déploiement multi-environnement.
	Jenkins : [16] Automatisation des tests, compilation et déploiement via des pipelines CI/CD personnalisés.
	Node.js : [17] Environnement JavaScript côté serveur, adapté aux applications temps réel, API REST et microservices.
	MongoDB & PostgreSQL : [18] [19] MongoDB pour les données non relationnelles ; PostgreSQL pour les données relationnelles structurées.
	Flutter : [20] Développement mobile multiplateforme avec une seule base de code Dart, compatible Android/iOS.

2.7 Conclusion

En conclusion, ce chapitre a établi les bases solides de notre projet en définissant les acteurs, les besoins fonctionnels et non fonctionnels, et en établissant un Product Backlog détaillé. Les diagrammes UML ont permis une meilleure compréhension de l'architecture de l'application et le choix des technologies appropriées a été déterminant pour garantir une application performante et sécurisée. Les outils et l'environnement de travail sélectionnés ont renforcé l'efficacité de notre processus de développement. Cela nous positionne bien pour les étapes de mise en œuvre qui seront discutées dans les chapitres suivants.

Conclusion Générale

2.8 Synthèse du Projet

Au terme de ce projet de fin d'études, nous avons réussi à développer une plateforme complète dédiée aux projets académiques et professionnels. À travers sept sprints consécutifs, notre équipe a progressivement construit un écosystème numérique cohérent et performant, alliant fonctionnalités sociales, professionnelles et technologiques avancées.

L'approche par microservices adoptée dès le début du projet s'est avérée particulièrement adaptée, permettant une évolution modulaire et une intégration fluide de nouvelles fonctionnalités. Cette architecture a également facilité le développement parallèle des différentes composantes du système, optimisant ainsi notre cycle de développement tout en maintenant un haut niveau de qualité et de cohérence.

2.9 Récapitulatif des Réalisations

Lors des deux premiers sprints, nous avons établi les fondations solides de notre plateforme en implémentant un système d'authentification robuste avec différents rôles utilisateurs, une gestion complète des profils professionnels, un mécanisme de relations et de suivis entre utilisateurs, ainsi qu'une interface administrateur pour la gestion des comptes. Ces développements initiaux ont constitué le socle sur lequel l'ensemble des fonctionnalités ultérieures ont été construites, garantissant sécurité et personnalisation de l'expérience utilisateur.

Les sprints 3 et 4 ont enrichi la plateforme de sa dimension sociale et analytique à travers un système complet de publication et de partage de contenus (posts et stories), des mécanismes d'interactions sociales (likes, commentaires), un tableau de bord administrateur avec visualisation de métriques, et un système de notifications en temps réel. Ces fonctionnalités ont transformé notre plateforme en un véritable réseau professionnel interactif, favorisant l'engagement des utilisateurs et offrant des outils analytiques précieux pour le pilotage.

Le cinquième sprint a introduit les dimensions professionnelles clés de notre plateforme : un système de publication et de recherche d'offres d'emploi, des outils de gestion de projets collaboratifs, des mécanismes de candidature et de suivi, et une recherche avancée et filtrée des opportunités. Ces développements ont considérablement étendu la portée de notre application, en offrant de réelles

opportunités professionnelles à ses utilisateurs.

Le sixième sprint s'est concentré sur l'intégration d'un système de réunions vidéo complet avec l'intégration de BigBlueButton pour les visioconférences, des fonctionnalités avancées de partage d'écran et de documents, l'enregistrement et archivage des sessions, ainsi que la programmation et gestion des réunions. Cette étape a ajouté une dimension de communication synchrone essentielle, complétant les canaux de collaboration asynchrones déjà présents sur la plateforme.

Le dernier sprint a marqué une avancée technologique majeure avec l'intégration de capacités d'intelligence artificielle : un système de génération de questions d'entretien adaptées au domaine professionnel, un matching intelligent entre CV et offres d'emploi, une architecture RAG pour des résultats contextualisés et pertinents, et le déploiement local du modèle Mistral 7B via Ollama. Ces fonctionnalités d'IA ont considérablement enrichi l'expérience utilisateur, offrant des outils d'aide à la décision et de préparation professionnelle innovants.

2.10 Architecture Technique

L'architecture technique déployée pour notre plateforme se distingue par sa modularité et sa robustesse. L'architecture microservices permet d'encapsuler chaque fonctionnalité majeure dans un service indépendant et spécialisé, communiquant via des APIs standardisées. L'API Gateway centralisée constitue un point d'entrée unique assurant le routage, la sécurité et la gestion des requêtes vers les différents microservices.

Nous avons opté pour des bases de données polyglotes avec l'utilisation de MongoDB pour les données non structurées (profils, posts), PostgreSQL pour les données relationnelles (authentification, notifications), et Qdrant pour les vecteurs d'embeddings (IA). La communication temps réel est assurée par l'intégration de WebSockets et Socket.IO pour les fonctionnalités instantanées comme les notifications, le chat et les réunions.

La conteneurisation Docker permet un déploiement unifié et portable de l'ensemble des services, facilitant la scalabilité et la maintenance. Enfin, des mécanismes de sécurité transversale avec l'authentification JWT uniforme à travers tous les services et la validation des données à chaque niveau protègent l'intégrité du système. Cette architecture a permis d'atteindre un équilibre optimal entre performance, maintenabilité et évolutivité, offrant une base solide pour les évolutions futures de la plateforme.

2.11 Technologies et Innovations

Notre projet a intégré un large éventail de technologies modernes, reflétant notre volonté d'innovation et d'excellence technique. Du côté backend, nous avons utilisé Node.js et Express.js comme fondation pour la plupart des microservices, Fastify pour les services nécessitant des performances optimales, GraphQL pour une communication client-serveur flexible et efficace, Python et FastAPI pour le microservice d'intelligence artificielle, MongoDB et PostgreSQL comme bases de données principales, et Redis pour le cache et les sessions.

Pour le frontend, nous avons misé sur React.js pour l'interface utilisateur dynamique et réactive, Redux pour la gestion d'état globale, Chart.js pour la visualisation des données analytiques, Socket.IO pour les communications temps réel, et TypeScript pour un code frontend typé et robuste.

Notre projet se distingue également par plusieurs innovations techniques significatives : l'intégration de BigBlueButton pour les visioconférences, le déploiement local de modèles d'IA avec Ollama, l'architecture RAG pour l'enrichissement contextuel des réponses du LLM, l'utilisation d'une base de données vectorielle Qdrant pour la recherche sémantique, et la mise en place d'un système de notifications en temps réel basé sur GraphQL Subscriptions.

2.12 Défis Relevés

Le développement d'une plateforme aussi complète a nécessité de surmonter plusieurs défis majeurs. Nous avons dû maintenir une cohérence de l'expérience utilisateur malgré l'architecture en microservices, assurant ainsi une interface fluide et unifiée. Les enjeux de performance et de scalabilité ont été relevés grâce à des mécanismes de cache, l'optimisation des requêtes et l'architecture distribuée, permettant d'excellentes performances même avec une augmentation de la charge.

La sécurité des données a constitué une priorité, avec l'implémentation de mesures robustes à tous les niveaux pour protéger les informations sensibles des utilisateurs. L'intégration de l'IA a représenté un défi technique particulier, le déploiement local de Mistral 7B ayant nécessité des optimisations importantes pour fonctionner avec des ressources limitées. Enfin, la communication interservices a été fluidifiée par la standardisation des échanges et l'utilisation de l'API Gateway, résolvant ainsi les complexités inhérentes à l'architecture microservices.

2.13 Perspectives d'Évolution

Si ce projet a déjà atteint un niveau de maturité avancé, plusieurs pistes d'évolution se dessinent pour l'avenir. Nous envisageons de mettre en place un système d'apprentissage continu de

l'IA avec un mécanisme de feedback pour améliorer progressivement les modèles. Le développement d'applications mobiles natives pour iOS et Android permettrait d'accroître l'accessibilité de la plateforme.

L'expansion de l'analytique avec l'intégration d'outils d'analyse prédictive offrirait de nouvelles perspectives pour anticiper les tendances du marché de l'emploi. L'internationalisation de la plateforme pourrait l'adapter à différents marchés et langues pour une portée globale. Des connecteurs vers des plateformes tierces comme LinkedIn et GitHub enrichiraient l'écosystème de services professionnels. Enfin, un système de recommandations avancé exploitant l'IA permettrait de suggérer des connexions, formations et opportunités professionnelles encore plus pertinentes pour chaque utilisateur.

2.14 Apport Personnel et Professionnel

Ce projet de fin d'études a représenté une expérience extrêmement enrichissante, tant sur le plan technique que professionnel. Il nous a permis d'approfondir nos compétences en architecture microservices, développement full-stack et intégration d'IA, constituant ainsi une véritable maîtrise technique dans des domaines de pointe. L'application concrète des méthodes agiles et des principes de développement itératif a renforcé notre capacité de gestion de projet, compétence essentielle dans l'environnement professionnel moderne.

La collaboration au sein d'une équipe pluridisciplinaire, avec répartition des tâches et synchronisation régulière, a développé nos aptitudes au travail d'équipe. Une veille technologique constante nous a permis d'explorer et d'intégrer des technologies émergentes, notamment dans le domaine de l'IA. Enfin, ce projet nous a amenés à développer une vision produit globale, alliant considérations techniques et besoins utilisateurs, compétence stratégique fondamentale dans le développement de solutions numériques.

2.15 Mot de la Fin

Au terme de ce projet, nous sommes fiers d'avoir relevé le défi de concevoir et réaliser une plateforme professionnelle complète, moderne et innovante. Les sept sprints successifs ont permis d'aboutir à un produit qui répond pleinement aux objectifs initiaux tout en intégrant des fonctionnalités avancées comme l'intelligence artificielle.

L'architecture microservices adoptée a prouvé sa pertinence tout au long du développement, offrant la flexibilité nécessaire pour faire évoluer chaque composante du système indépendamment.

L'intégration cohérente de ces services à travers une API Gateway centrale a permis de maintenir une expérience utilisateur fluide et unifiée malgré la complexité technique sous-jacente.

Ce projet constitue non seulement l'aboutissement de notre formation académique, mais aussi un véritable tremplin vers notre vie professionnelle future, nous ayant permis de développer des compétences techniques pointues et transversales, particulièrement recherchées sur le marché du travail actuel.

Bibliographie

- [1] ruiji cloud - container platform. <https://www.ruijicloud.com/>, 2026. (Accessed on 1/02/2026).
- [2] Cisco meraki - container platform. <https://www.CiscoMeraki.com/>, 2026. (Accessed on 1/02/2026).
- [3] Ubiquiti unifi - container platform. <https://www.UbiquitiUniFi.com/>, 2026. (Accessed on 1/02/2026).
- [4] Zabbix - container platform. <https://www.Zabbix.com/>, 2026. (Accessed on 1/02/2026).
- [5] Ken Schwaber and Jeff Sutherland. *The Scrum Guide – The Definitive Guide to Scrum : The Rules of the Game*. 2020. (Accessed on 29/05/2025).
- [6] Unified modeling language (uml). <https://www.uml.org/>. (Accessed on 27/02/2025).
- [7] Nextjs - framework for the web. <https://nextjs.org>. (Accessed on 27/02/2025).
- [8] Docker compose - define and run multi-container docker applications. <https://docs.docker.com/compose/>, 2025. (Accessed on 29/05/2025).
- [9] Flask - lightweight wsgi web application framework. <https://flask.palletsprojects.com/en/stable/>. (Accessed on 27/02/2025).
- [10] Ollama - run large language models locally. <https://ollama.ai/>. (Accessed on 27/02/2025).
- [11] Mistral ai - open source language models. <https://mistral.ai/>. (Accessed on 27/02/2025).
- [12] Github - code hosting platform. <https://github.com/>, 2025. (Accessed on 29/05/2025).
- [13] Gitlab - the devops platform. <https://about.gitlab.com/>. (Accessed on 27/02/2025).
- [14] Visual studio code - code editing. <https://code.visualstudio.com/>. (Accessed on 27/02/2025).
- [15] Docker - container platform. <https://www.docker.com/>, 2025. (Accessed on 29/05/2025).
- [16] Jenkins - automation server. <https://www.jenkins.io/>, 2025. (Accessed on 29/05/2025).

- [17] What is node.js? - definition from whatis.com. <https://whatis.techtarget.com/definition/Nodejs>. (Accessed on 27/02/2025).
- [18] Mongodb : The developer data platform. <https://www.mongodb.com/>. (Accessed on 27/02/2025).
- [19] Postgresql : The world's most advanced open source relational database. <https://www.postgresql.org/>. (Accessed on 27/02/2025).
- [20] Flutter - build apps for any screen. <https://flutter.dev/>, 2025. (Accessed on 29/05/2025).

Résumé :

Ce projet vise à développer une plateforme web centralisée pour la supervision et le contrôle des réseaux Ruijie via une interface unique. Elle permet de visualiser la topologie des projets et d'assurer un suivi en temps réel des équipements pour optimiser la gestion, jusque-là dispersée. La solution intègre un module d'IA pour la détection d'anomalies, ainsi que des systèmes de mailing et de publicité interne pour les alertes et la communication. Développée avec Vite.js/React, NestJS et MongoDB, la plateforme assure une sécurité par rôles et la génération automatisée de rapports techniques.

Mots clés : Supervision centralisée , Réseaux Ruijie , Topologie , IA , Machine Learning , Temps réel , NestJS , React , MongoDB , Mailing , Publicité interne.

Abstract :

This project aims to develop a centralized web platform for the supervision and control of Ruijie networks through a single interface. It enables the visualization of project topology and ensures real-time equipment monitoring to optimize previously dispersed management. The solution integrates an AI module for anomaly detection, along with mailing and internal advertising systems for alerts and communication. Developed using Vite.js/React, NestJS, and MongoDB, the platform ensures role-based security and the automated generation of technical reports.

Keywords : Centralized Supervision, Ruijie Networks, Network Topology, AI , Machine Learning, Real-time Monitoring, NestJS, React, MongoDB, Mailing System, Internal Advertising.